00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 07

Sıralamanın Ötesi

Öbek Sıralaması, Doğrusal Zamanlı Sıralama ve Ara Proje

MODÜL II · ARAMA VE SIRALAMA

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** **Öbek ve Doğrusal Sıralama**
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan üç soru

- **S1 (H6).** n = 8 farklı elemanı sıralayan karşılaştırmalı bir algoritma en kötü durumda en az kaç karşılaştırma yapar?
- **S2 (H5).** (70, Ada) (85, Banu) (70, Cem) nota göre **kararlı** sıralanırsa çıktı nedir?
- **S3 (AP-1 H10).** 0–100 notlarının sıklığını `sayac[not]++` ile sayan dizi kaç elemanlıdır?

Yanıtlar: üç fikir bu haftanın üç aracı

- **S1:** $\lceil \log_2 8! \rceil = \lceil \log_2 40\,320 \rceil =$ **16**; birleştirmeli sıralamanın en kötüsü 17, araya eklemenin 28 (Kavram 4).
- **S2:** (70, Ada) (70, Cem) (85, Banu). Kararsız bir algoritma iki 70'liyi ters sırada verebilir: öbek sıralaması tam bunu yapacak.
- **S3:** 101 eleman (0 … 100). `sayac[not]` değeri **indis** olarak kullanır: sayma sıralamasının bütün fikri.
- Bu hafta: alt sınırın **kapsamı**, kararlılığın **zorunlu** olduğu bir algoritma (taban) ve değeri indis yapan sıralama.

Büyük Soru: karşılaştırmadan sıralamak?

18,7 ms

sayma sıralaması, 10^6 anahtar (0 … $10^6 - 1$)

142,0 ms

C kütüphanesi qsort, aynı girdi

0

anahtar karşılaştırması (sayma)

- Karşılaştırma yapmadan sıralamak mümkün mü ve Ω(n log n) sınırı nasıl “aşılır”? Alt sınır yanlış değil: yalnızca “a < b mi?” diye soran algoritmalar içindir.

GERÇEK DÜNYADAN

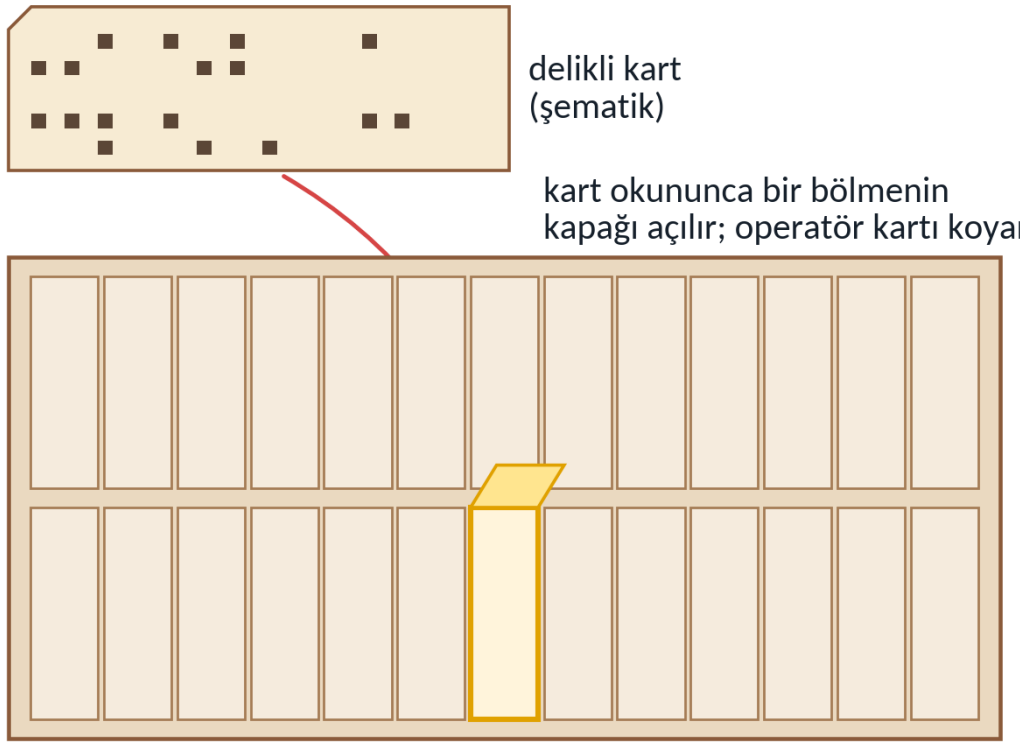
1888 · 1890 · 1936

Kartları sütun sütun ceplere ayırmak

- 1888 yarışması: 1880 verisinden dört St. Louis bölgesi **72,5 saatte** kaydedildi (rakipler 144,5 ve 100,5 saat).
- 1890 tabülatörleri: **40 sayaç kadranı**, yanında bir sıralama masası; deneyimli memur dakikada **80 kart**.
- İlk sıralama kutusunda **26 bölme**: kart okununca bir kapak açılır, operatör kartı koyar.
- **Önce birler sütunu**, kartları sırayla topla, sonra onlar: yöntem en eski IBM kılavuzunda (1936); büyük olasılıkla adı bilinmeyen operatörlerin buluşu.

U.S. Census Bureau (2024). The Hollerith machine; Knuth, D. E. (1998). TAOCP Vol. 3, § 5.5.

Hollerith'in sıralama kutusu (şematik) ve zaman çizgisi



sıralama kutusu: 26 bölme (2 sıra × 13; şematik)

- **1888** — Yarışma: 1880 verisinden dört St. Louis bölgesi; kayıt 72,5 sa (rakipler 144,5 ve 100,5), tabülasyon hazırlığı 5,5 sa (44,5 ve 55,5)
- **1890** — Sayım ≈ 100 Hollerith makinesiyle; tabülatörde 40 sayaç kadranı, yanında sıralama masası; dakikada 80 kart
- **1901, 1904** — Hollerith'in patentli sıralayıcıları: modern kart sıralayıcılarına benzer
- **1930** — LSD yönteminin bilinen ilk basılı değinisi: Comrie'nin makalesinde bir not
- **1936** — Yöntem bilinen en eski IBM sıralayıcı kılavuzunda: önce birler sütunu
- **1950'ler** — Büroda yerini bilgisayarlara bırakır

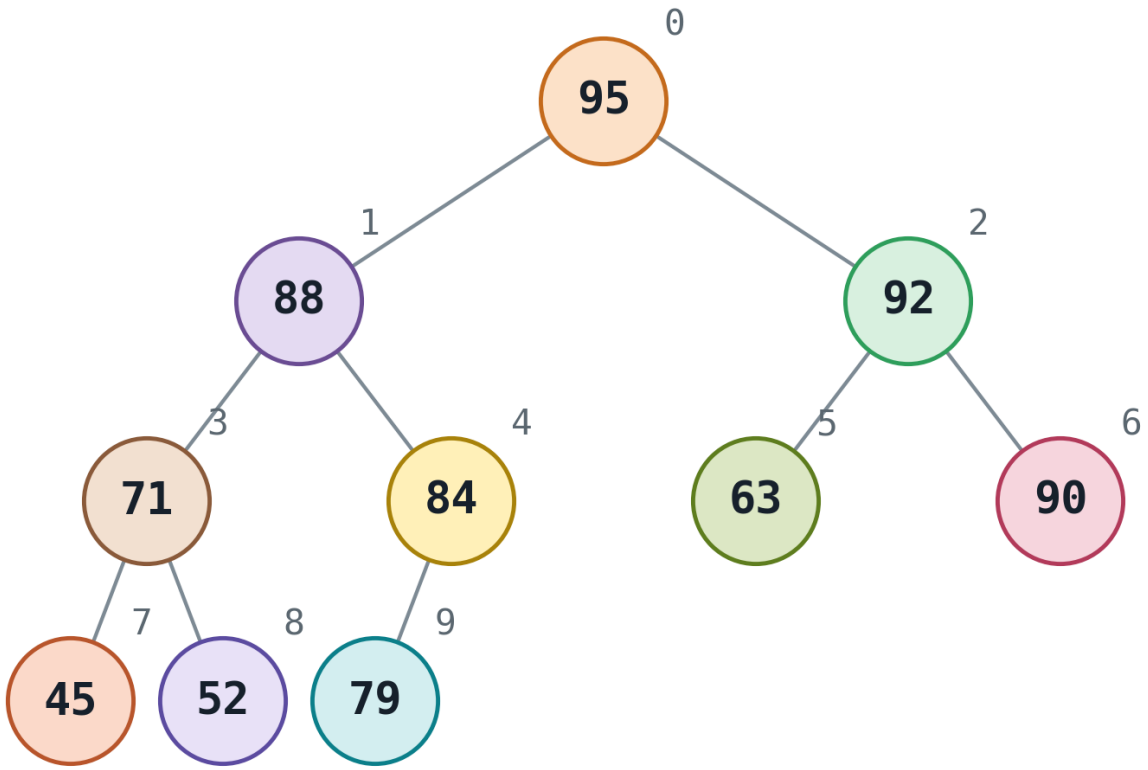
Kaynak: U.S. Census Bureau (2024); Knuth (1998, § 5.5). Çizim şematiktir; ölçü ve yerleşim temsilîdir.

ÖĞRENME ÇIKTILARI

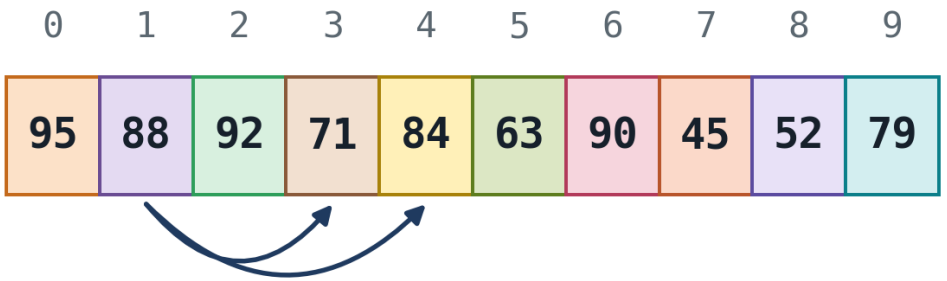
Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 7.1	Tam ikili ağacın dizi gösterimini (ebeveyn ve çocuk indisleri) kullanarak ikili öbek işlemlerini (ekleme, en büyüğü çıkarma, yukarı/aşağı süzme) gerçekler.
ÖÇ 7.2	Aşağıdan yukarıya öbek kurmanın O(n), öbek sıralamasının O(n log n) olduğunu açıklar; öbek sıralamasını gerçekler.
ÖÇ 7.3	Öncelik kuyruğunu ilk-k ve çok yollu birleştirme problemlerine uygular.
ÖÇ 7.4	Sayma ve taban (LSD) sıralamasını gerçekler; kararlılığın taban sıralamasının doğruluğundaki rolünü açıklar.
ÖÇ 7.5	Veri özelliklerine (boyut, anahtar aralığı, neredeyse sıralılık, kararlılık gereksinimi, bellek) göre uygun sıralama algoritmasını gerekçeli olarak seçer.

Tam ikili ağaç bir dizide: işaretçi yok



aynı renk = aynı eleman
indis kutunun üstünde



sol(i) = 2i + 1
sağ(i) = 2i + 2
ebeveyn(i) = (i − 1) / 2

sol(i) = 2i + 1 = (i << 1) | 1 · sağ(i) = 2i + 2 · ebeveyn(i) = (i − 1) / 2 (0 tabanlı) · aynı renk = aynı eleman

TAHMİN ET

a[4] = 84'ün ebeveyni ve çocukları hangileri? (n = 10)

A ebeveyn a[2]; çocuklar a[9] ve a[10]

B ebeveyn a[1]; çocuklar a[8] ve a[9]

C ebeveyn a[2]; tek çocuk a[9]

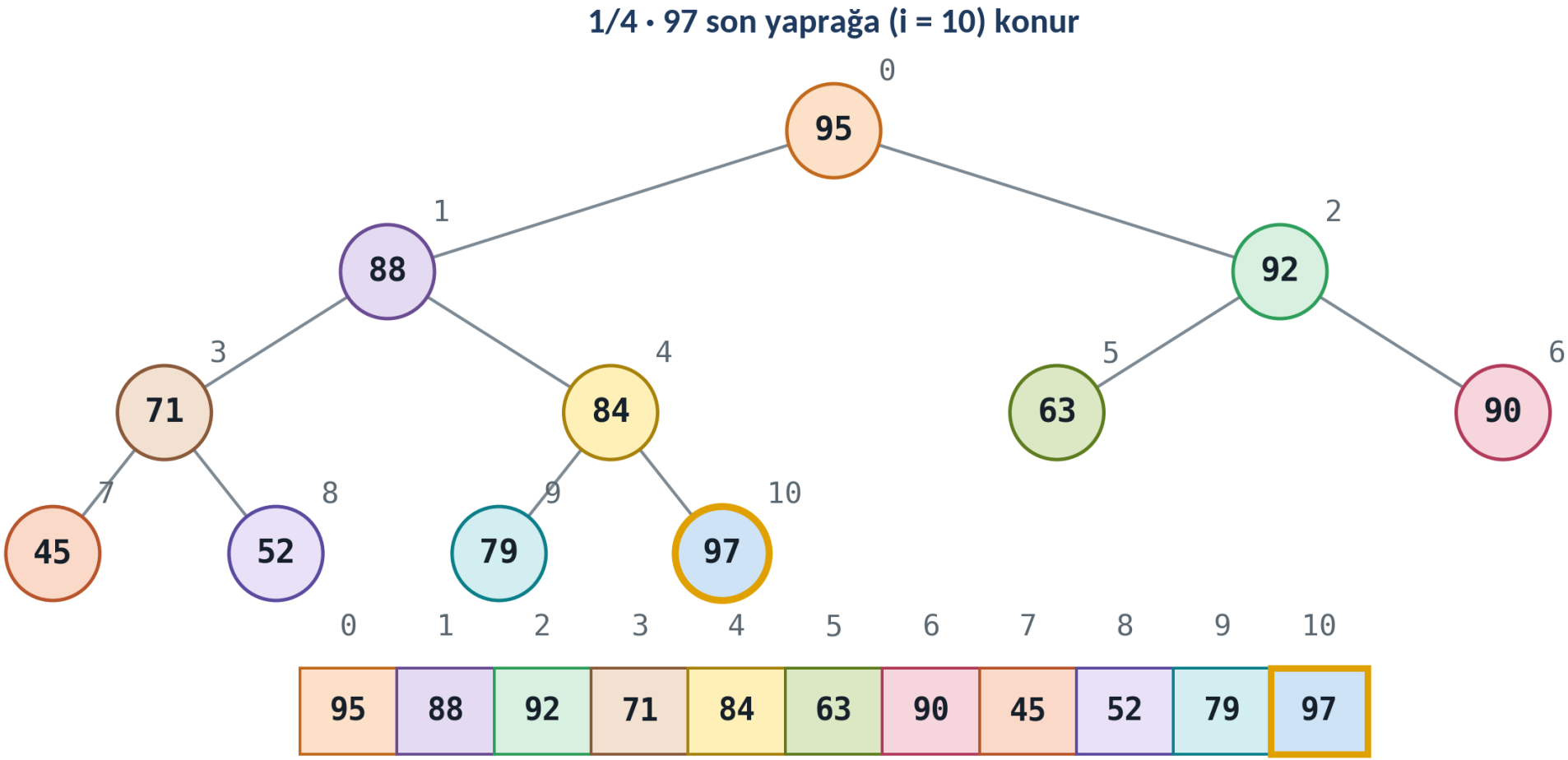
D ebeveyn a[1]; tek çocuk a[9] = 79

Öncelik kuyruğu: arayüz ve üç gerçekleştirim

Öbek özelliği: her i > 0 için **a[ebeveyn(i)] ≥ a[i]** (Örnek 2 ilk ihlali arar). Kök en büyüktür; kardeşler arasında sıra yoktur.

İşlem	Anlamı	Sırasız dizi	Sıralı dizi	İkili öbek
`ekle(x)`	önceliğiyle kuyruğa koy	O(1)	O(n)	O(log n)
`en_buyuk()`	en öncelikliyi gör	O(n)	O(1)	O(1)
`en_buyugu_cikar()`	en öncelikliyi al ve sil	O(n)	O(1)	O(log n)
`kur(a)`	n elemandan kuyruk	O(1)	O(n log n)	Θ(n)

Ekleme: 97 ilk boş yaprağa (a[10]) konur



Biçim korunur (tam ikili ağaç); bozulan yalnızca 97 ile ebeveyni arasındaki sıra

TAHMİN ET

97 köke ulaşana dek kaç takas yapılır?

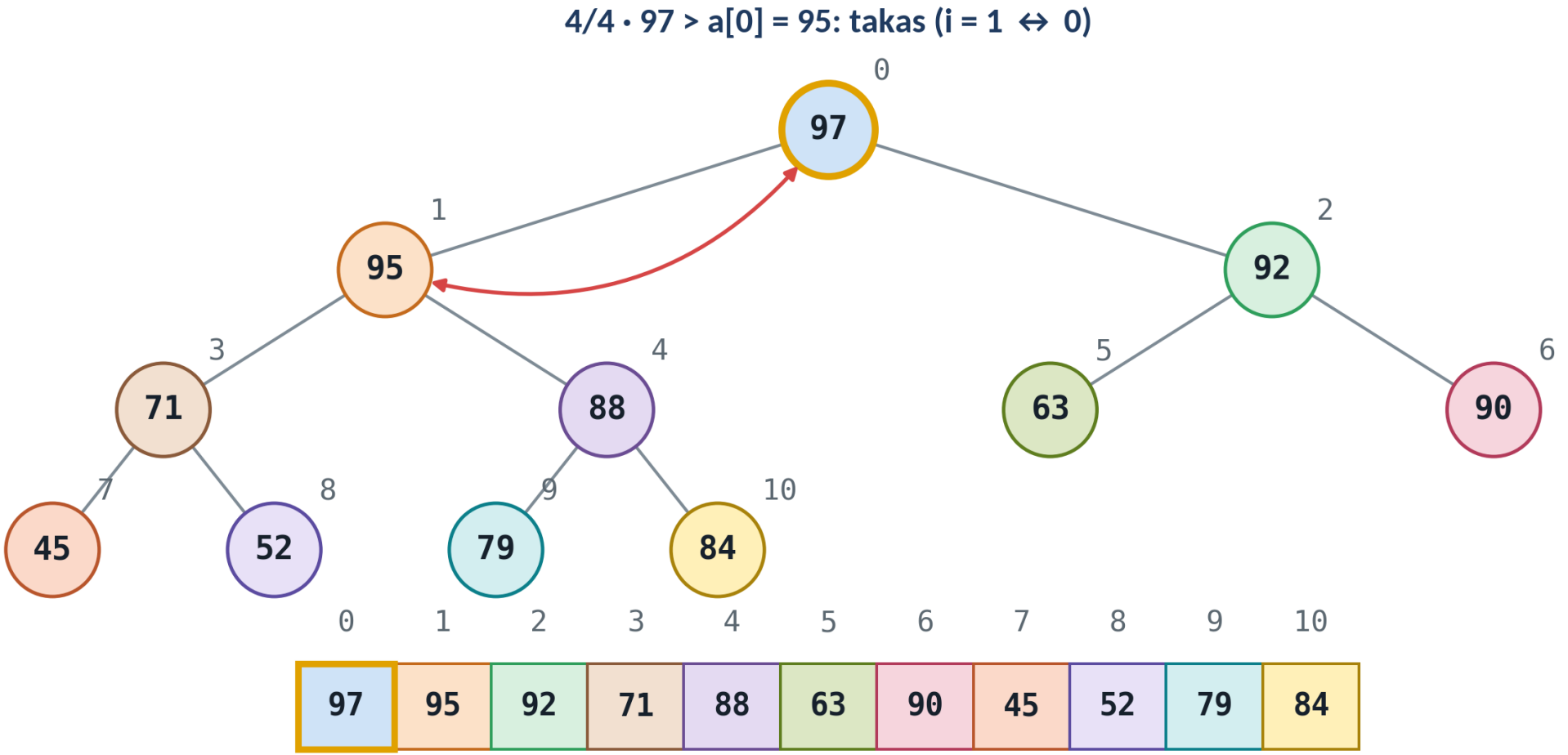
A 1

B 2

C 3

D 4

Yukarı süzme (sift-up): üç takas, 97 kökte



ekle 97: konum 10 -> 0, takas = 3 (düzey sayısı - 1) [97 95 92 71 88 63 90 45 52 79 84]

TAHMİN ET

Kök 97 çıkarıldı, son yaprak 84 köke taşındı: [84 95 92 71 88 63 90 45 52 79]. 84 ilk adımda kiminle yer değiştirir?

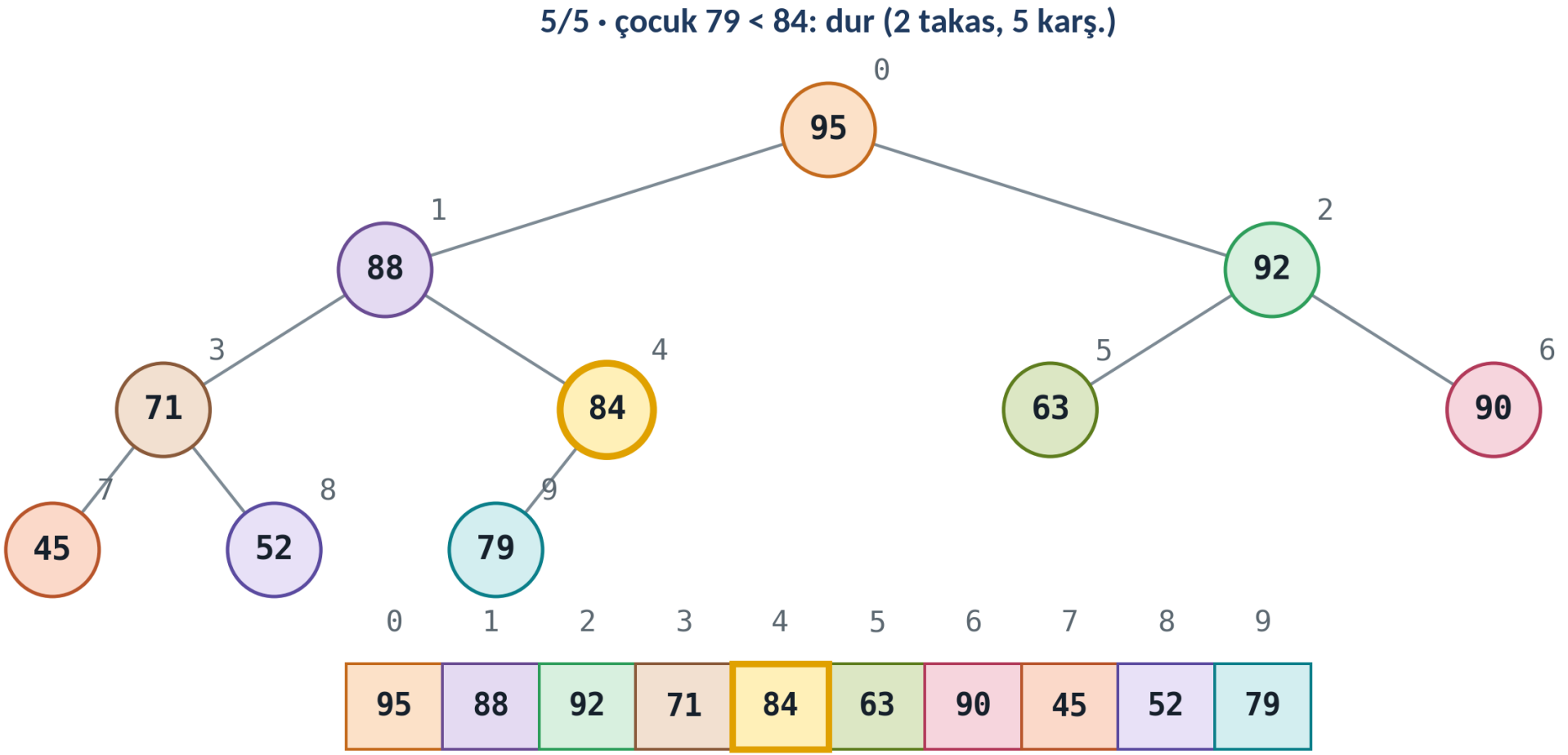
A 95 (sol çocuk, büyük olan)

B 92 (sağ çocuk)

C 71 (en küçük torun)

D Hiçbiriyle: öbek zaten geçerli

Aşağı süzme (sift-down): büyük çocukla takas, çocuk küçükse dur



çıkar -> 97 (köke 84 taşındı), karşılaştırma = 5, takas = 2 · düzey başına en çok 2 karşılaştırma

Süzme: iki döngü, iki değişmez

```
algoritma YUKARI-SÜZ(a, i)
    iken i > 0 ve a[i] > a[(i − 1) / 2]
        a[i] ⇔ a[(i − 1) / 2];  i ← (i − 1) / 2
algoritma AŞAĞI-SÜZ(a, n, i)
    iken 2i + 1 < n
        c ← 2i + 1
        eğer c + 1 < n ve a[c + 1] > a[c] ise c ← c + 1
        eğer a[c] ≤ a[i] ise dur
        a[i] ⇔ a[c];  i ← c
```

- **Yukarı:** özellik yalnız i ile ebeveyni arasında bozuk olabilir.
- **Aşağı:** yalnız i ile çocukları arasında; i'nin iki alt ağacı öbektir.
- Her ikisi O(log n): en çok yükseklik kadar düzey.
- Ekleme: a[n] ← x; YUKARI-SÜZ. Çıkarma: a[0] ← a[n − 1]; AŞAĞI-SÜZ.

KOD

C: aşağı süzme (Örnek 3)

c/H07_03_ekle_cikar.c · satır 48–62

C

```c
48  static int asagi_suz(int *a, int n, int i)
49  {
50      while (2 * i + 1 < n) {
51          int c = 2 * i + 1;  /* sol çocuk */
52          if (c + 1 < n && buyuk_mu(a[c + 1], a[c])) {
53              c++;  /* sağ çocuk daha büyük */
54          }
55          if (!buyuk_mu(a[c], a[i])) {
56              break;  /* öbek özelliği sağlandı */
57          }
58          takas(&a[i], &a[c]);
59          i = c;
60      }
61      return i;
62  }
```

- 52: sağ çocuk varsa ve **büyükse** c ona geçer.
- 55: büyük çocuk bile küçük-eşitse **dur**.
- Sayaçlar `buyuk_mu` içinde: karşılaştırmayı programla sayıyoruz.
- Test: on eklemeden sonra `Örnek 1'in öbeğiyle aynı mı? evet`; sırayla çıkarılanlar azalan.

Üç dilde: yukarı süzmenin gövdesi

H07_03_ekle_cikar.c — C

```c
    while (i > 0 && buyuk_mu(a[i], a[ebeveyn(i)])) {
        takas(&a[i], &a[ebeveyn(i)]);
        i = ebeveyn(i);
    }
    return i;
```

H07_03_ekle_cikar.cpp — C++

```cpp
    while (i > 0 && buyuk_mu(a[i], a[ebeveyn(i)])) {
        takas(a[i], a[ebeveyn(i)]);
        i = ebeveyn(i);
    }
    return i;
```

H07_03_ekle_cikar.py — Python

```python
    while i > 0 and buyuk_mu(a[i], a[ebeveyn(i)]):
        takas(a, i, ebeveyn(i))
        i = ebeveyn(i)
    return i
```

Gövde üç dilde aynı yapıda (aşağı süzme de öyle; Ek A.3); fark öbeğin boyutunu kimin tuttuğunda: C'de ayrı `n`, C++'ta `std::vector`, Python'da liste. (satırlar: C 40–44 · C++ 39–43 · Python 35–38)

ALGORİTMA KİMLİK KARTI

En büyüğü çıkarma (aşağı süzme) *extract-max (sift-down)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kökü al, son yaprak köke, aşağı süz	En büyük öbek, n ≥ 1	O(1)	Θ(log n)	Θ(log n)	O(1)

Özellikler	Düzey başına en çok 2 karşılaştırma ve 1 takas; taşınan yaprak küçük olduğundan çoğunlukla dibe iner
Değişmez / doğruluk	Aşağı süzmede öbek özelliği yalnızca i ile çocukları arasında bozuk olabilir; i'nin iki alt ağacı öbektir
Kütüphane karşılığı	C: yok (elle) · C++: `std::priority_queue::top` + `pop`, `std::pop_heap` · Python: `heapq.heappop` (en küçüğü çıkarır)
Ne zaman kullanılır	En öncelikli işi tekrar tekrar almak; öbek sıralaması; k yollu birleştirme
Ne zaman kullanılmaz	Hem en küçük hem en büyük sık gerekiyorsa (iki öbek; arama ağacı)

TAHMİN ET

Artan girdi [45 52 63 71 79 84 88 90 92 95] aşağıdan yukarıya öbek kurulurken toplam kaç takas yapılır?

A 5

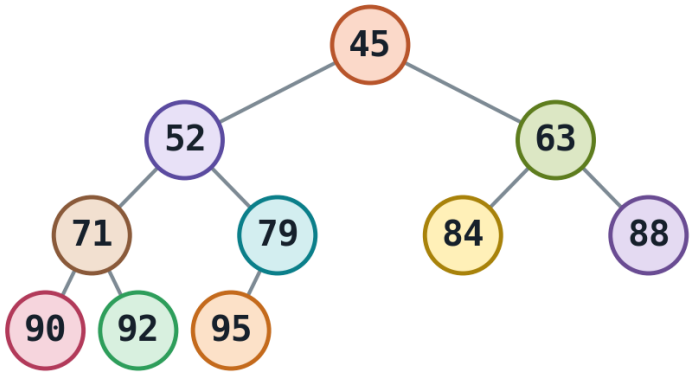
B 8

C 10

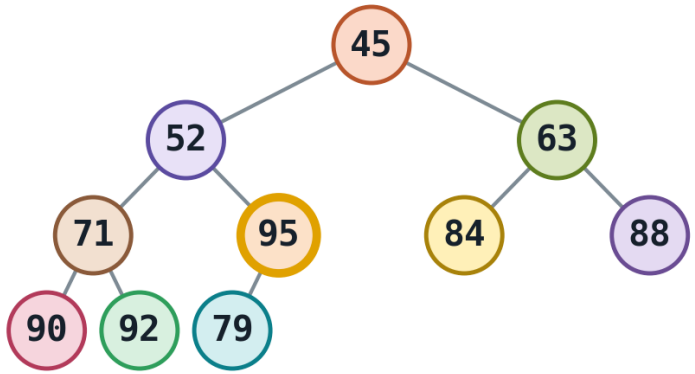
D 19

Aşağıdan yukarıya kurma: yapraklar zaten öbek

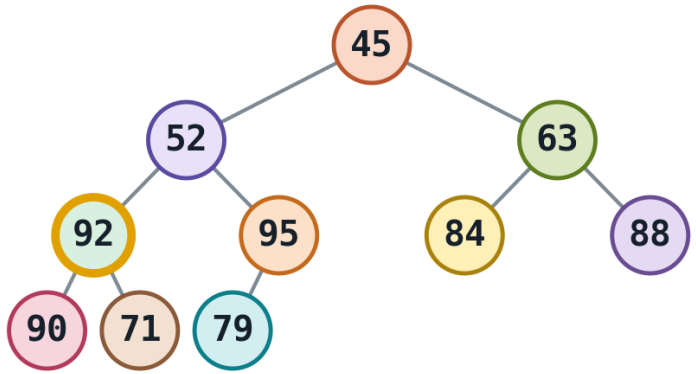
1) girdi (artan): yapraklar 5..9 zaten öbek



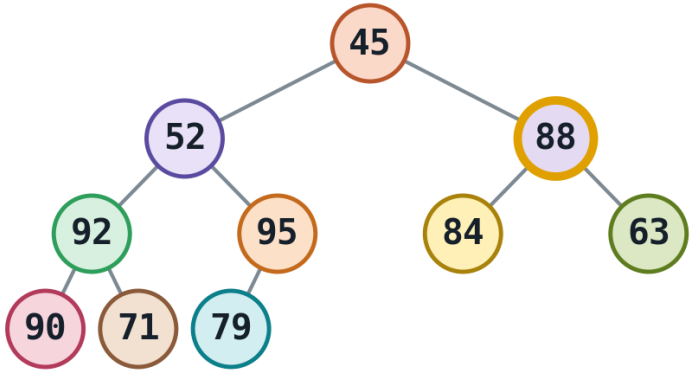
2) i = 4 süzüldü: 1 takas



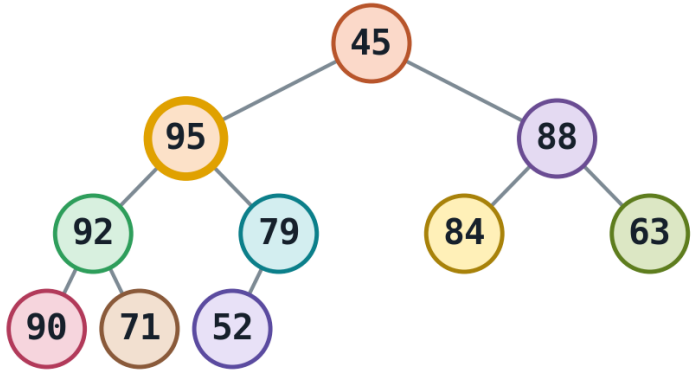
3) i = 3 süzüldü: 1 takas



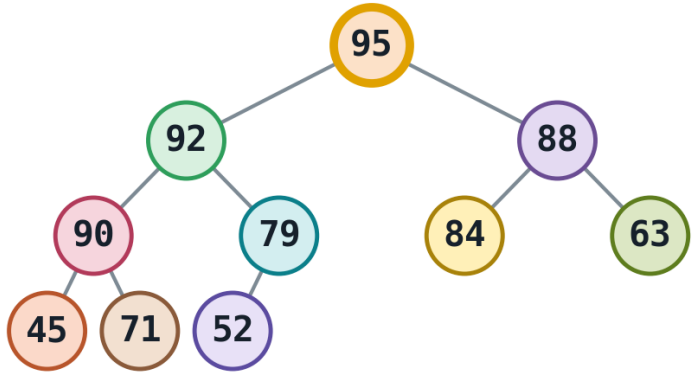
4) i = 2 süzüldü: 1 takas



5) i = 1 süzüldü: 2 takas

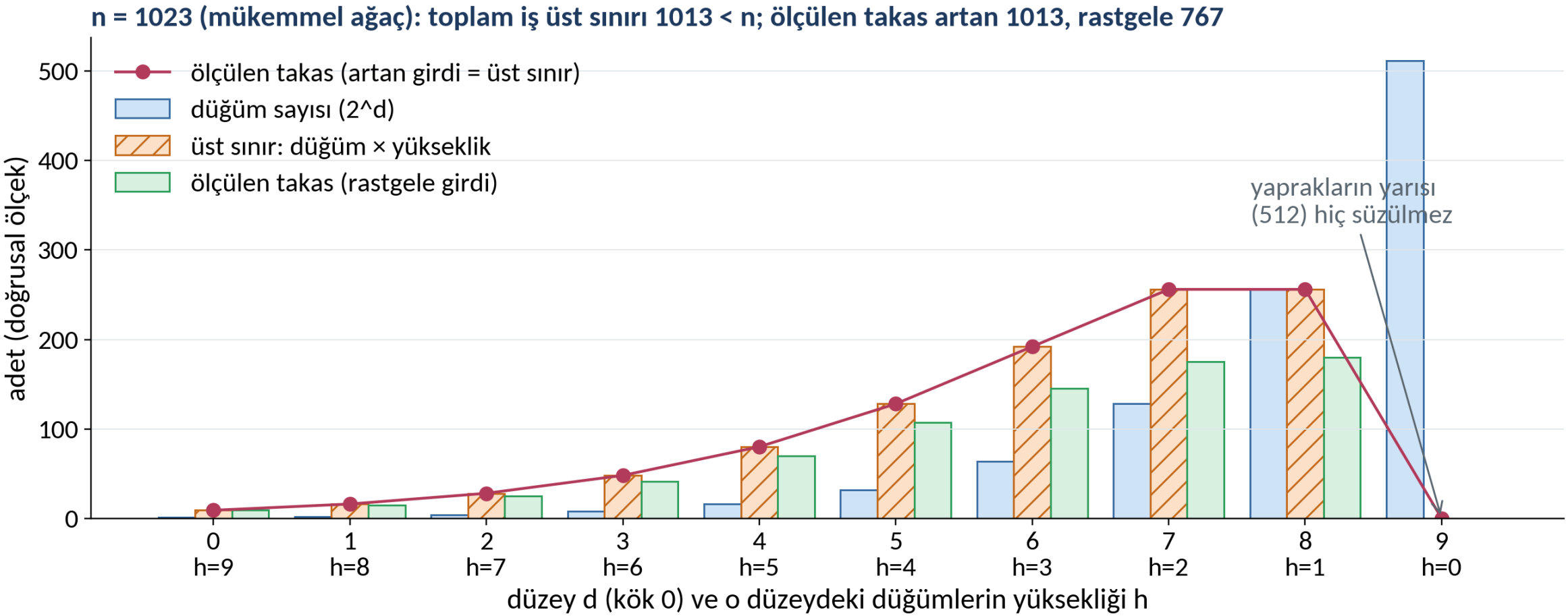


6) i = 0 süzüldü: 3 takas



i = ⌊n/2⌋ − 1'den 0'a her düğüm aşağı süzülür; sarı: o adımda süzülen düğüm

Neden O(n)? Düzey aşağı indikçe iş azalır



n = 1023: üst sınır Σ yükseklik = 1 013, artan girdi 1 013, rastgele 767 takas — hepsi < n

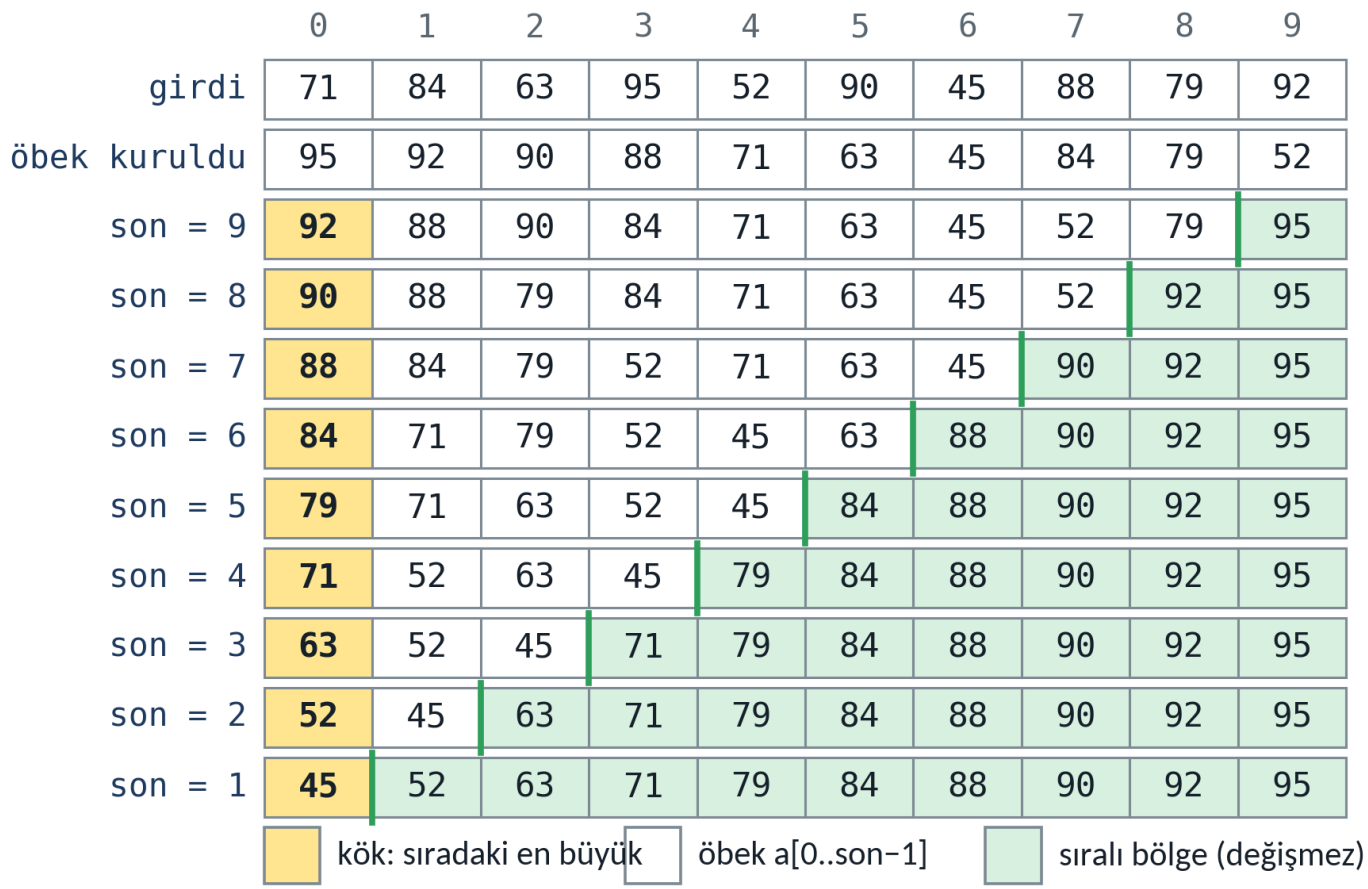
ALGORİTMA KİMLİK KARTI

Aşağıdan yukarıya öbek kurma *bottom-up heap construction (heapify)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Son iç düğümden köke aşağı süz	Yok: herhangi bir dizi	Θ(n)	Θ(n)	Θ(n)	O(1)

Özellikler	Takas ≤ Σ yükseklik = n − popcount(n) < n; düğümlerin yarısı yapraktır, hiç süzülmez
Değişmez / doğruluk	i'den sonraki her düğüm (i + 1, …, n − 1) bir öbeğin köküdür; i süzülünce i de öyle olur
Kütüphane karşılığı	C: yok (elle) · C++: `std::make_heap` (en çok 3n karşılaştırma) · Python: `heapq.heapify` (en küçük öbek, doğrusal)
Ne zaman kullanılır	Öbek sıralamasının ilk evresi; bütün elemanlar baştan biliniyorsa
Ne zaman kullanılmaz	Elemanlar akış hâlinde geliyorsa (tek tek ekleme kullanılır)

Öbek sıralaması: kök sona, sıralı bölge büyür



karşılaştırma = 40, takas = 28 (öbek kurma dahil) · yeşil: sıralı bölge (değişmez şeridi)

Öbek sıralaması: kur, sonra n − 1 kez kökü sona al

```
algoritma ÖBEK-SIRALAMASI(a, n)
    ÖBEK-KUR(a, n)
    son ← n − 1'den 1'e kadar
        a[0] ⇔ a[son]
        AŞAĞI-SÜZ(a, son, 0)
```

c/H07_05_obek_siralamasi.c · satır 59–66

C

```c
static void obek_siralamasi(int *a, int n)
{
    obek_kur(a, n);
    for (int son = n - 1; son > 0; son--) {
        takas(&a[0], &a[son]);  /* en büyük yerine */
        asagi_suz(a, son, 0);  /* kalan öbeği onar */
    }
}
```

Θ(n log n) her girdide · O(1) ek bellek · kararsız · uyarlanabilir değil

TAHMİN ET

Öbek sıralaması (70 Ada)(85 Banu)(70 Cem)(90 Defne)(85 Emre)(70 Filiz)(85 Gül) kayıtlarını nota göre sıralıyor. 70'ler hangi sırada çıkar?

A Ada, Cem, Filiz (girdi sırası)

B Cem, Ada, Filiz

C Ada, Filiz, Cem

D Filiz, Cem, Ada

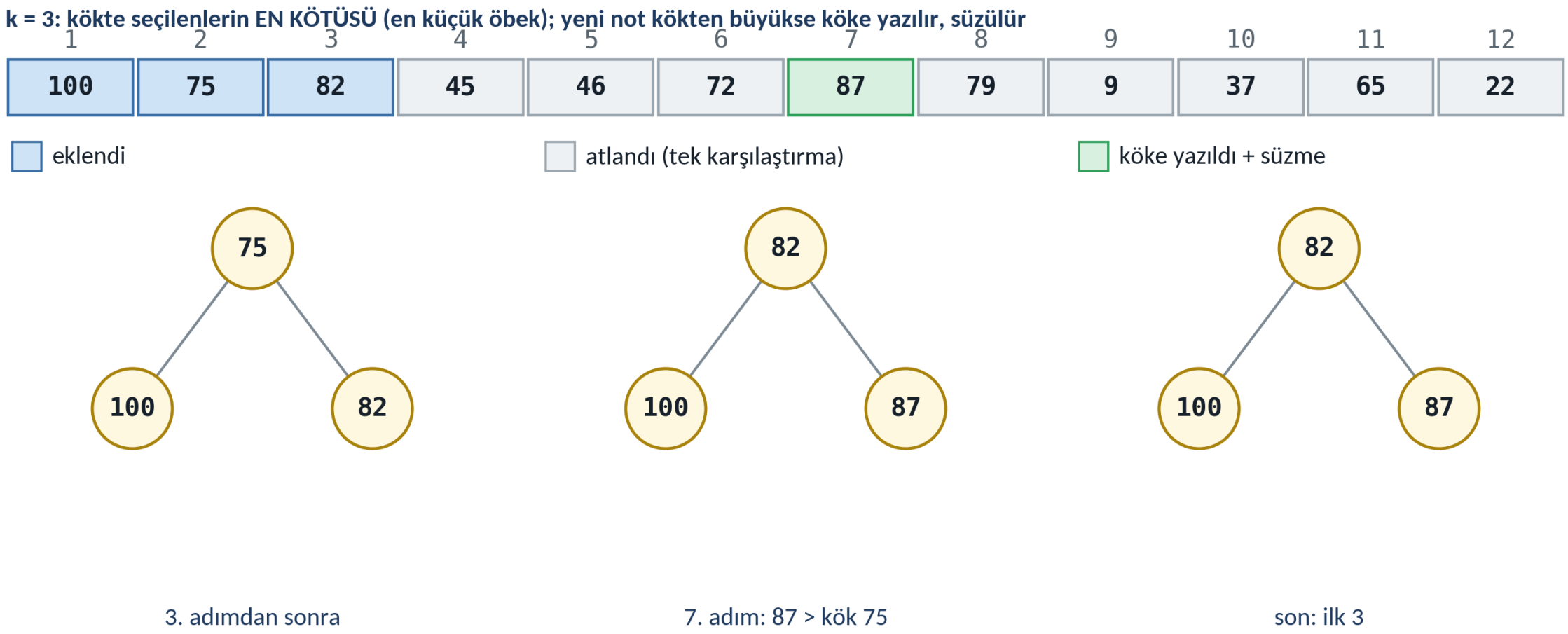
ALGORİTMA KİMLİK KARTI

Öbek sıralaması *heapsort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Öbek kur; kökü sona al, süz	Karşılaştırılabilir anahtar	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(1)

Özellikler	Kararsız · yerinde · uyarlanabilir değil · önbellek dostu değil (uzak indislere sıçrar): bu makinede hızlı sıralamanın ≈ 2 katı süre (oturuma göre 1,75–2,52)
Değişmez / doğruluk	a[son+1..n−1] sıralıdır ve a[0..son] öbeğindeki her elemandan büyük-eşittir; a[0..son] en büyük öbektir
Kütüphane karşılığı	C: yok (`qsort` algoritmayı belirtmez) · C++: `std::make_heap` + `std::sort_heap`; `std::partial_sort` · Python: `heapq` ile (heapify + n kez heappop; yerinde değil)
Ne zaman kullanılır	En kötü durum O(n log n) ve O(1) ek bellek birlikte gerekiyorsa; introsort'un yedeği
Ne zaman kullanılmaz	Kararlılık ya da pratik hız önemliyse; neredeyse sıralı veride

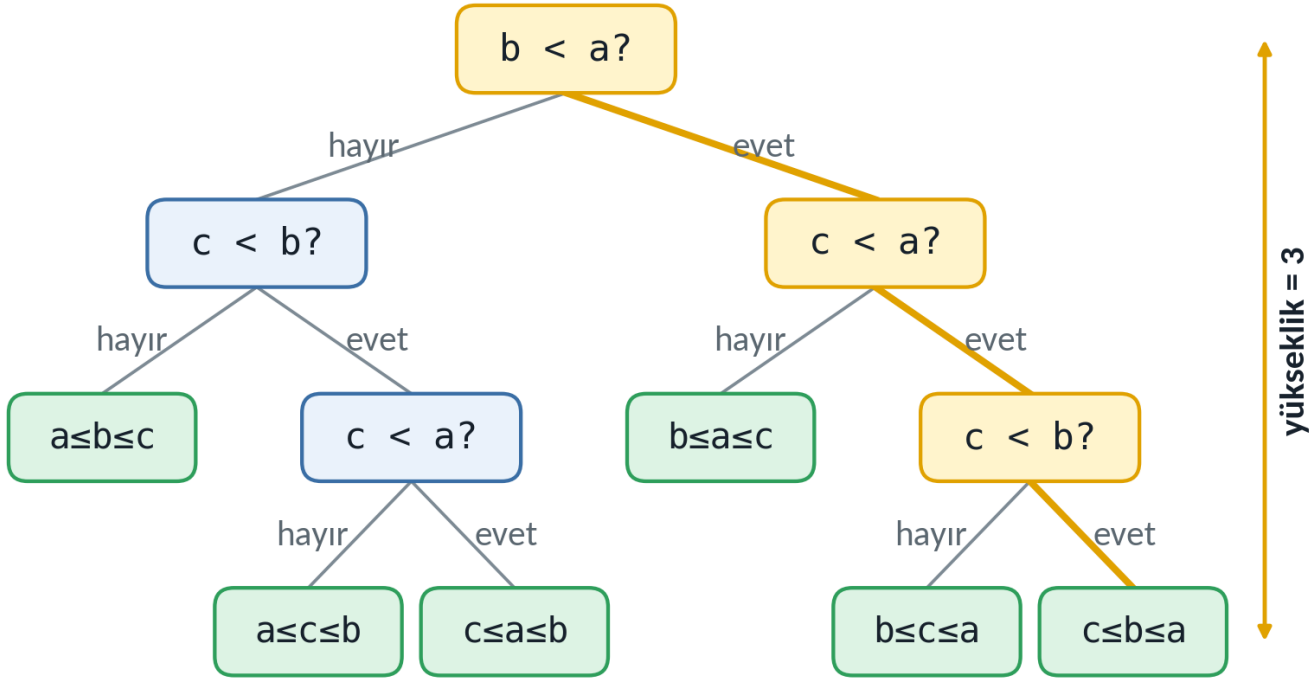
Öncelik kuyruğuyla ilk-k: kökte seçilenlerin en kötüsü



Boyutu k olan en küçük öbek: O(n log k), O(k) bellek · n = 10^5, k = 10: 100 205 karş. ($n \log_2 n$ = 1 660 964) · Örnek 7: k liste, O(N log k)

Karşılaştırmalı sıralama: en az ⌈log₂ n!⌉ karşılaştırma

n = 3: 6 yaprak (3! sıralama) · en kötü 3 karşılaştırma = ⌈log₂ 3!⌉



n	n!	⌈log₂ n!⌉	araya (en kötü)	birleştirmeli (en kötü)
1	1	**0**	0	0
2	2	**1**	1	1
3	6	**3**	3	3
4	24	**5**	6	5
5	120	**7**	10	8
6	720	**10**	15	11
7	5040	**13**	21	14
8	40320	**16**	28	17

$\log_2 n! / (n \log_2 n)$: 0,856 (10^3) · 0,913 (10^5) → $\Omega(n \log n)$

n! düzenin her biri ayrı yaprak → $2^h \geq n!$ → yükseklik $h \geq \log_2 n! \approx n \log_2 n$

Alt sınır “aşılmaz”: kapsamın dışına çıkılır

Sayma

`sayac[a[i]]++` tek adımda **k + 1 yoldan** birine dallanır. Koşul: anahtar tamsayı, aralık küçük ($k \lesssim n$).

Taban (LSD)

Anahtar **d basamağa** bölünür, her basamakta kararlı sayma. Koşul: sabit uzunlukta anahtar.

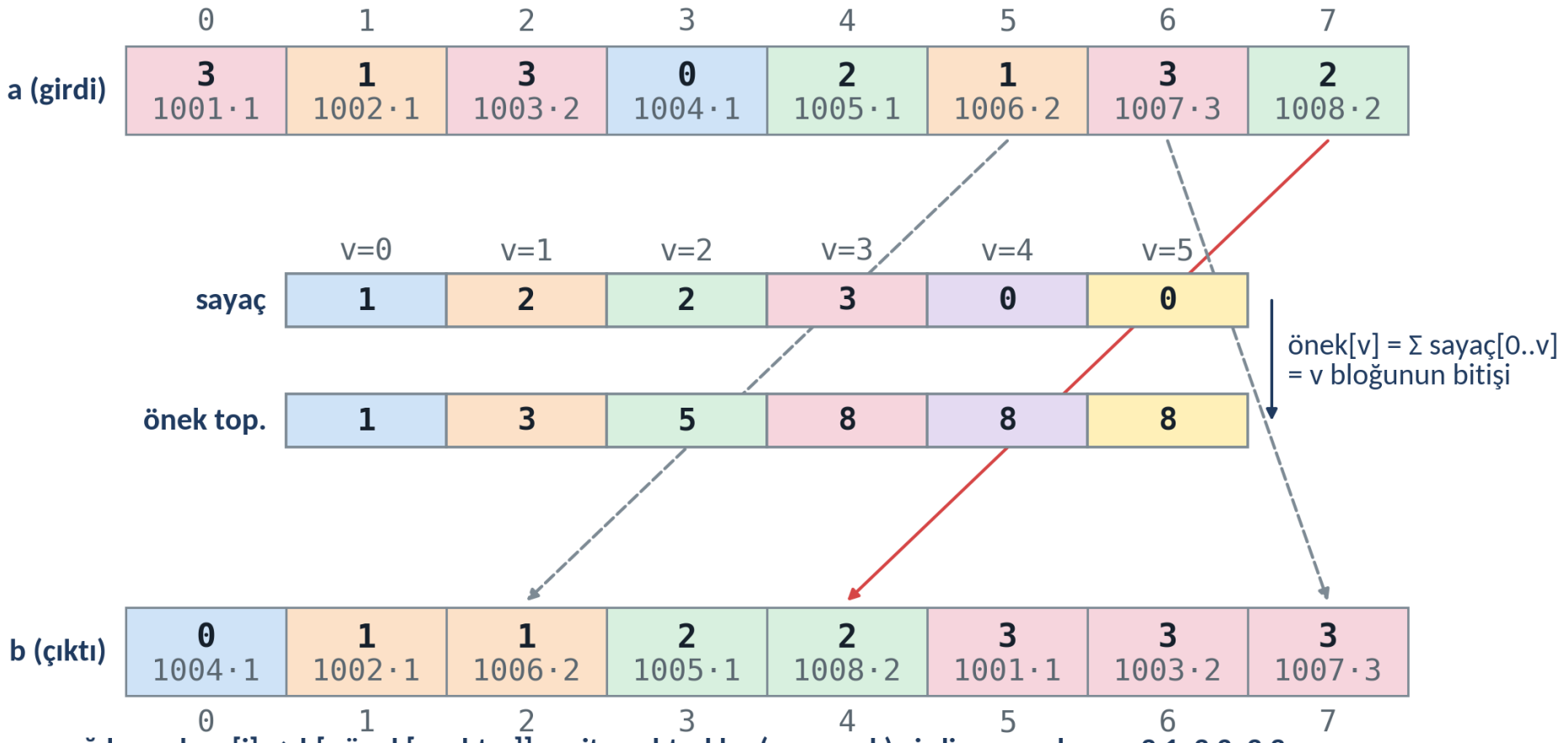
Kova

Değer aritmetikle **kova numarasına** çevrilir. Koşul: [0, 1)'de yaklaşık düzgün dağılım.

Ω(n) kalır

Her elemanı en az bir kez okumadan **hiçbir** algoritma sıralayamaz; doğrusal sıralamalar bu sınıra ulaşır.

Kararlı sayma: sayaç → önek toplam → sağdan sola



sağdan sola: a[i] → b[--önek[anahtar]] · eşit anahtarlılar (aynı renk) girdi sırasını korur: 3·1, 3·2, 3·3

önek[v] = anahtarı ≤ v olan kayıt sayısı = v bloğunun bitişi; eşitler girdi sırasını korur (3·1, 3·2, 3·3)

TAHMİN ET

Sağdan sola yerleştirmede a[5] = (1006, 1) sırası geldiğinde kayıt b'nin neresine yazılır? (önek: 1 3 5 8 8 8)

A b[0]

B b[1]

C b[2]

D b[3]

Üç dilde: sayaç dizisi ve sayma

H07_08_sayma_siralamasi.c — C

```c
    int *sayac = calloc((size_t)k + 1, sizeof *sayac);
    if (sayac == NULL) {
        exit(1);
    }
    for (int i = 0; i < n; i++) {
        sayac[a[i]]++;  /* değer = indis */
        adim_sayisi++;
    }
```

H07_08_sayma_siralamasi.cpp — C++

```cpp
    std::vector<int> sayac(k + 1, 0);  // sıfırlı
    for (int x : a) {
        sayac[x]++;  // değer = indis
        adim_sayisi++;
    }
```

H07_08_sayma_siralamasi.py — Python

```python
    sayac = [0] * (k + 1)  # sıfırlı
    for x in a:
        sayac[x] += 1  # değer = indis
        adim_sayisi += 1
```

C'de `calloc` (sıfırlanmış bellek, sonra `free`), C++'ta `std::vector<int>(k + 1, 0)`, Python'da `[0] * (k + 1)` (yuva başına 8 bayt). (satırlar: C 25–32 · C++ 25–29 · Python 17–20)

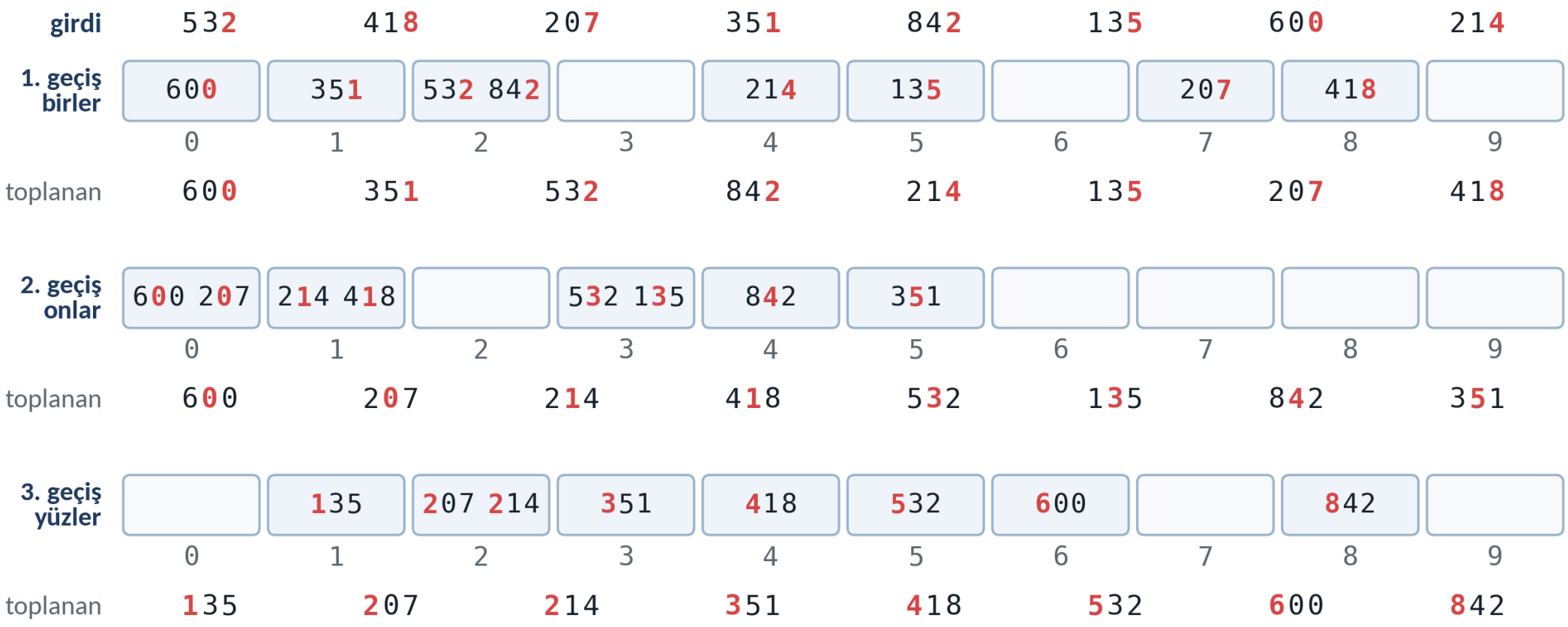
ALGORİTMA KİMLİK KARTI

Sayma sıralaması *counting sort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Değer = indis: sıklık + önek toplam	Anahtarlar 0..k tamsayı	Θ(n + k)	Θ(n + k)	Θ(n + k)	O(n + k)

Özellikler	Kararlı (önek + sağdan sola) · karşılaştırma yok · k ≫ n ise maliyeti k belirler
Değişmez / doğruluk	Yerleştirme döngüsünde önek[v], v anahtarlı ve henüz yerleştirilmemiş kayıtların son hedef konumunun bir fazlasıdır
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`collections.Counter` yalnızca sayar)
Ne zaman kullanılır	Notlar, yaşlar, baytlar; not dağılımı (histogram); taban sıralamasının her geçişi
Ne zaman kullanılmaz	Geniş aralık (32/64 bit), tamsayı olmayan anahtar, bellek kısıtı

LSD taban sıralaması: Hollerith'in yöntemi



Her geçiş kırmızı basamağa göre kararlı sayma: 10 cebe dağıt, cepleri 0 → 9 sırayla topla

TAHMİN ET

207 ile 214'ün yüzler basamağı aynı (2). Üçüncü geçişten sonra hangisi önce gelir?

A 207: önceki geçişin sırası korunur

B 214: büyük olan sonda değil, önde biter

C Belirsiz: girdinin ilk sırasına bağlı

D Yan yana gelmezler

Üç dilde: bir basamakta kararlı sayma

H07_10_taban_siralamasi.c — C

```c
    for (int v = 1; v < 10; v++) {
        onek[v] += onek[v - 1];
    }
    /* sağdan sola: kararlı */
    for (int i = n - 1; i >= 0; i--) {
        b[--onek[a[i] / bolen % 10]] = a[i];
    }
```

H07_10_taban_siralamasi.cpp — C++

```cpp
    for (int v = 1; v < 10; v++) {
        onek[v] += onek[v - 1];
    }
    // sağdan sola: kararlı
    for (int i = n - 1; i >= 0; i--) {
        b[--onek[a[i] / bolen % 10]] = a[i];
    }
```

H07_10_taban_siralamasi.py — Python

```python
    for v in range(1, 10):
        onek[v] += onek[v - 1]
    # sağdan sola: kararlı
    for i in range(len(a) - 1, -1, -1):
        v = a[i] // bolen % 10
        onek[v] -= 1
        b[onek[v]] = a[i]
```

C/C++: `b[--onek[v]] = a[i]` tek deyim; Python'da `--` yok: önce azalt, sonra yaz. Bayt için `(x >> kaydir) & maske`. (satırlar: C 30–36 · C++ 34–40 · Python 21–27)

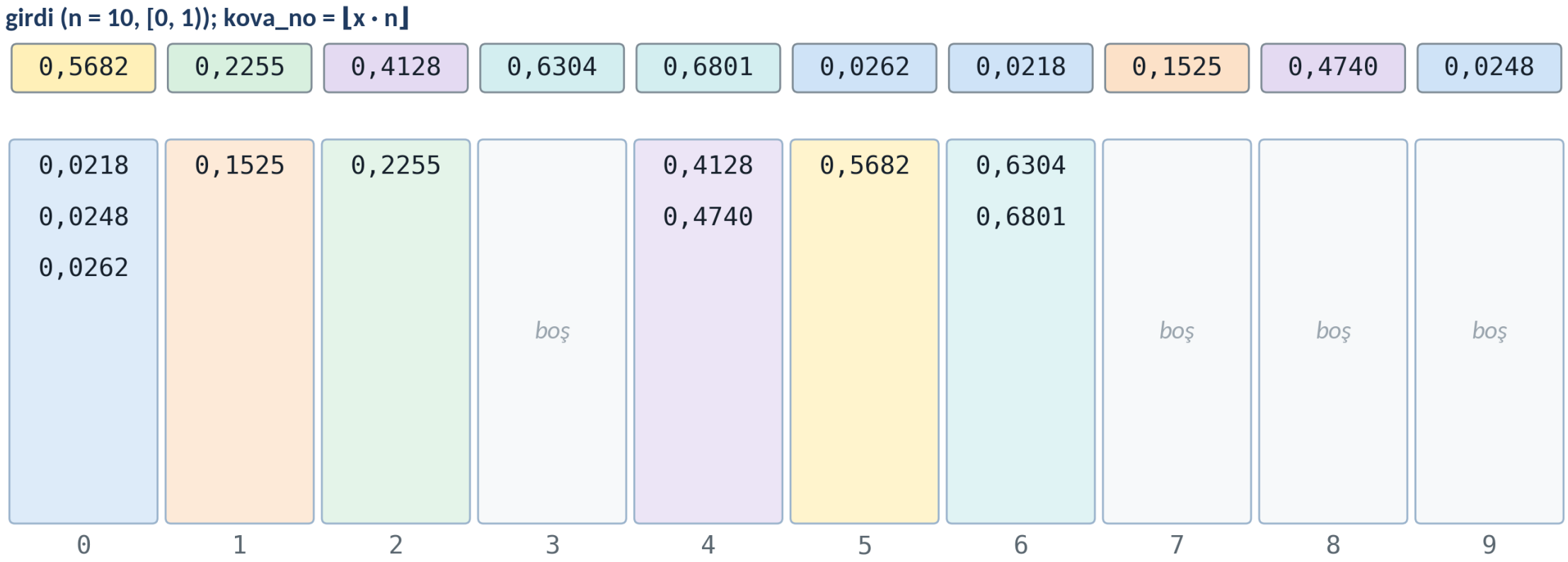
ALGORİTMA KİMLİK KARTI

Taban sıralaması (LSD) *LSD radix sort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
En düşük basamaktan kararlı sayma	Sabit uzunlukta anahtar (d, b)	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$O(n + b)$

Özellikler	Kararlı ara sıralama zorunlu · 32 bit, b = 256: 4 geçiş · taban seçimi bir ödünleşim
Değişmez / doğruluk	d. geçişten sonra dizi, anahtarların en düşük d basamağına göre sıralıdır (eşitlerde önceki sıra korunur)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	Çok sayıda sabit genişlikte tamsayı: öğrenci numarası, IP adresi
Ne zaman kullanılmaz	n küçükken; değişken uzunlukta ya da basamak yapısı olmayan anahtar

Kova sıralaması: dağılım varsayımına dayanır



kova içi araya ekleme sıralanır, kovalar sırayla birleştirilir · 4 boş kova, en dolu kova 3 eleman

Düzgün: eleman başına 0,453 karşılaştırma · tek kovada (n = 4000): 1009,0 — beklenen Θ(n), en kötü Θ(n²)

H5–H7: bütün sıralamalar tek tabloda

H	algoritma	en iyi	ortalama	en kötü	ek bellek	kararlı	yerinde	uyarlanabilir	10^6 rastgele (C -O2)
H5	seçmeli	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	hayır	evet	hayır	≈ 21,5 dk
H5	kabarcık (erken çıkışlı)	$O(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	evet	evet	evet	—
H5	araya ekleme	$O(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	evet	evet	evet	≈ 1,8 dk
H5	Shell	$O(n \log n)$	aralığa bağlı	$O(n\sqrt{n})$	$O(1)$	hayır	evet	evet	—
H6	birleştirmeli	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	evet	hayır	hayır	120,9 ms
H6	hızlı	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ yığın	hayır	evet	hayır	101,9 ms
H7	öbek	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	hayır	evet	hayır	207,2 ms
H7	sayma	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$O(n + k)$	evet	hayır	hayır	18,7 ms
H7	taban (LSD)	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$O(n + b)$	evet	hayır	hayır	20,3 ms
H7	kova	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$O(n)$	evet	hayır	hayır	—

Süreler bu makinede (işlemci zamanı ortancası); ≈ = ölçülmedi, n^2 modeliyle kestirim; — = deneyde yok. C kütüphanesi qsort: 142,0 ms. Kova: düzgün dağılım varsayımı; k = anahtar aralığı, d = geçiş, b = taban.

Karmaşıklıklar kimlik kartlarından; süre bu makinede C -O2, n = 10^6 rastgele (≈: kestirim)

TAHMİN ET

Bir nüfus kaydındaki 10^7 kişinin yaşı (0–120) artan sıralanacak. En uygun hangisi?

A Hızlı sıralama

B Sayma sıralaması

C Öbek sıralaması

D Kova sıralaması

Seçim rehberi: önce verinin profili

her soruda: evet → sağdaki seçim · hayır ↓ sonraki soru



Sorular karşılaştırmasız yöntemlerin koşullarıyla başlar; hiçbiri uymazsa kütüphane — önce ölçün

Atölyeler: öbek oyunu, senaryo kartları, kod incelemesi

Atölye 1 · Öbek oyunu (≈ 15 dk)

- On öğrenci on düğüm: önce dizi, sonra ağaç.
- 97 eklenir: 3 takas; kök çıkar: 2 takas.
- Artan on kartla kurma: 8 takas = n − popcount(n).

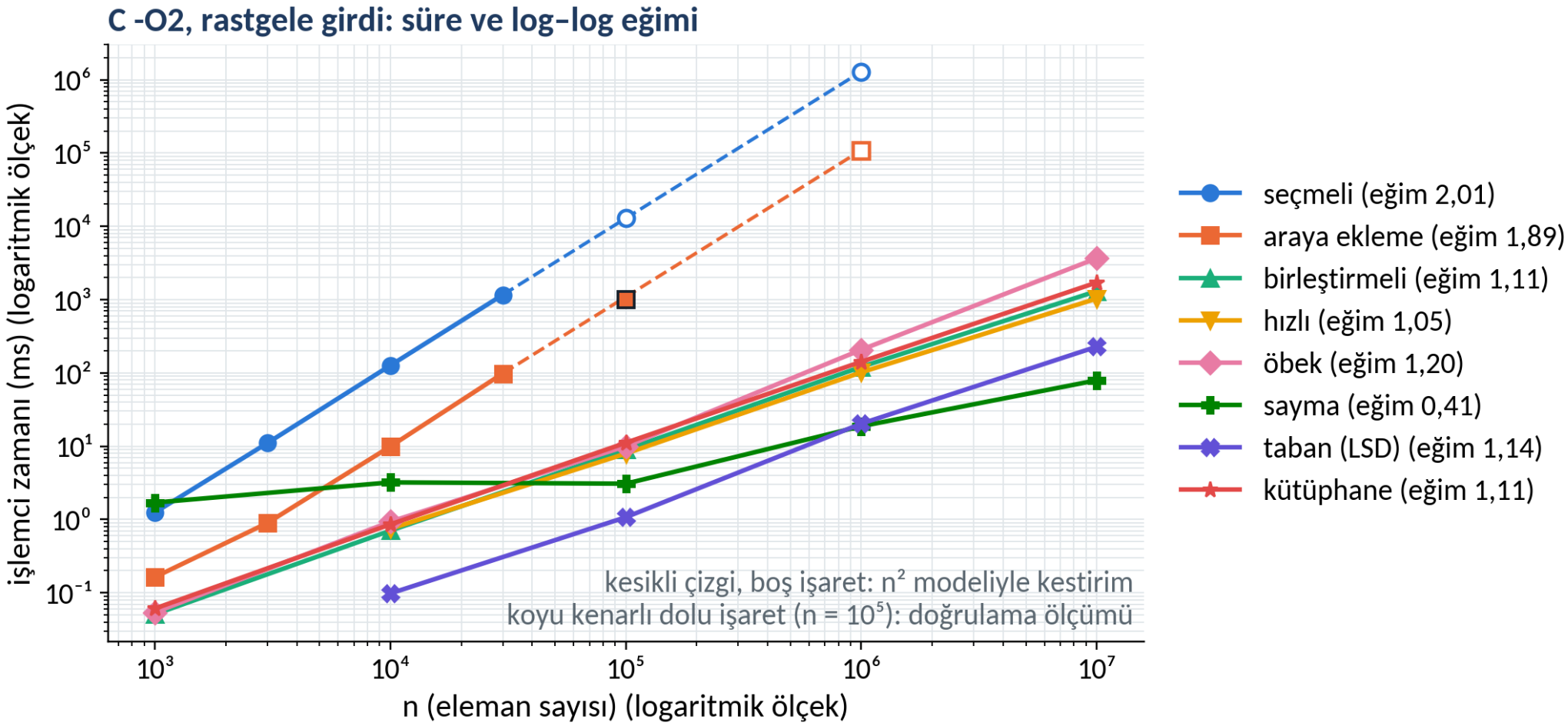
Atölye 2 · Sekiz senaryo (≈ 20 dk)

- Her grup iki kart: algoritma + ölçülebilir gerekçe.
- Ör. 10^7 yaş (0–120); bellek kısıtlı gömülü sistem.
- Gerekçeyi tablo ve deneyle sınayın: nasıl ölçerdik?

Atölye 3 · Ara proje (≈ 30 dk)

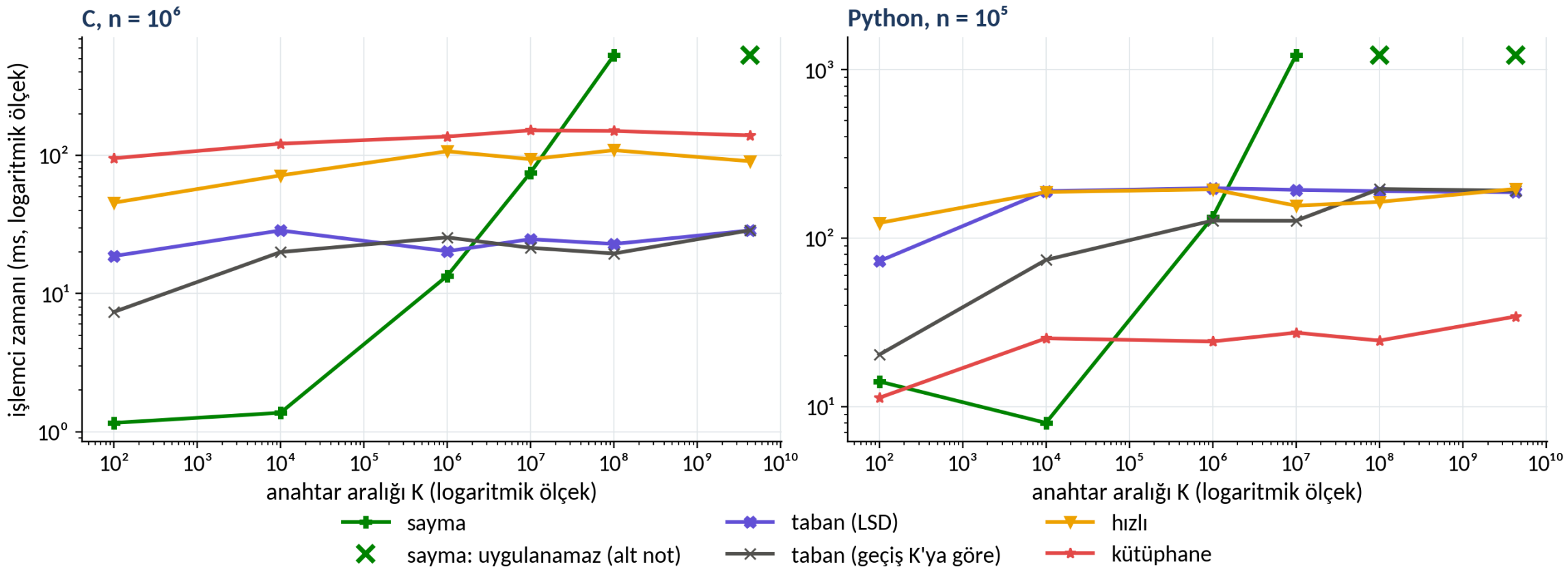
- v2.5 üzerinde aşağı süzme, kurma, sayma iskeleti.
- Eşli inceleme: sağ çocuk, size_t döngüsü, aralık dışı anahtar.
- Kararlılık testi ve ölçüm planı.

Haftanın Deneyi: log–log eğimleri



Girdiler: rastgele, sıralı, ters sıralı, çok tekrarlı (Örnek 12 kâhini: 32/32) · C -O2 rastgele eğimler: seçmeli 2,01, hızlı 1,05, öbek 1,20, taban 1,14

Anahtar aralığı: sayma K ile büyür, taban büyümez



×: sayma ölçülmedi. C'de K = 2^{32}: sayaç dizisi 16 GiB. Python'da K > 10^7 program reddeder (K = 10^8: sayaç listesi 763 MiB).

K = 2^{32}'de sayma uygulanamaz (16 GiB sayaç); taban sıralaması hep 4 geçiş, her geçişte 256 sayaç (1 KiB)

Hata Avcısı: yedi hata, gerçek belirtiler

#	Hata	Belirti (bu makinede)	Çözüm
1	0 / 1 tabanlı formülleri karıştırmak	azalan mı? HAYIR	(i − 1)/2, 2i + 1, 2i + 2 birlikte
2	Aşağı süzmede ilk çocuğu seçmek	öbek mi? HAYIR	önce büyük çocuğu bul
3	Öbek kurmayı i = 0'dan başlatmak	öbek mi? HAYIR	i = ⌊n/2⌋ − 1'den 0'a
4	Sayma: aralık dışı anahtar (−1, 105)	C: double free (UB); Python: IndexError	denetle ya da kaydır
5	Taban: kararsız ara geçiş	sıralı mı? HAYIR	sağdan sola yerleştir
6	heapq'yu en büyük öbek sanmak	en yüksek 3 not (sanılan): [45, 52, 63]	−x hilesi ya da nlargest
7	İşaretli sayılar taban sıralamasında	negatifler sonda: … 2147483647 -2147483648 -5 -1	işaret bitini çevir

Hata Avcısı 5: kararlı ve kararsız geçiş yan yana

	kararlı geçiş (sağdan sola yerleştirme)							
girdi	532	418	207	351	842	135	600	214
1. geçiş birler	600	351	532	842	214	135	207	418
2. geçiş onlar	600	207	214	418	532	135	842	351
3. geçiş yüzler	135	207	214	351	418	532	600	842

	kararsız geçiş (Hata Avcısı 5: soldan sağa)							
girdi	532	418	207	351	842	135	600	214
1. geçiş birler	600	351	842	532	214	135	207	418
2. geçiş onlar	207	600	418	214	135	532	842	351
3. geçiş yüzler	135	214	207	351	418	532	600	842

2. geçişte onlar basamağı eşit 600 ve 207 ters döndü; LSD önceki geçişin düzenine güvenir · son dizi sıralı mı? kararlı: evet, kararsız: HAYIR

Her geçiş "sıralı" görünür; eşit basamaklıların sırası ters dönünce sonraki geçişlerin dayandığı düzen kaybolur

Dil Penceresi: öncelik kuyruğu, sayaç, bit, kararlılık

Öğe	C	C++	Python
Öncelik kuyruğu	yok; elle (Kavram 7)	priority_queue: en BÜYÜK	heapq: en KÜÇÜK
En küçük ↔ en büyük	karşılaştırmayı çevir	std::greater<int>	−x hilesi, nlargest
Kurma / süzme	elle	make_heap, push_heap, pop_heap	heapify, heappush, heappop
Sayaç dizisi	calloc + free	vector<int>(k + 1, 0)	[0] * (k + 1)
Bayt alma	(x >> s) & 0xFF (uint32_t)	aynı	negatifte işaret genişler
Kararlı sıralama	standartta yok (qsort belirtmez)	stable_sort (sort değil)	sorted, list.sort

Mühendis Gözüyle · İleri Bakış · Merak

Mühendis Gözüyle

- Önce veriyi tanı: boyut, anahtar aralığı, düzen, kararlılık, bellek.
- Kütüphaneyi kullan, sözleşmesini oku (qsort/sort kararsız).
- Aralık dışı anahtar = güvenlik sorunu: denetle, ASan.

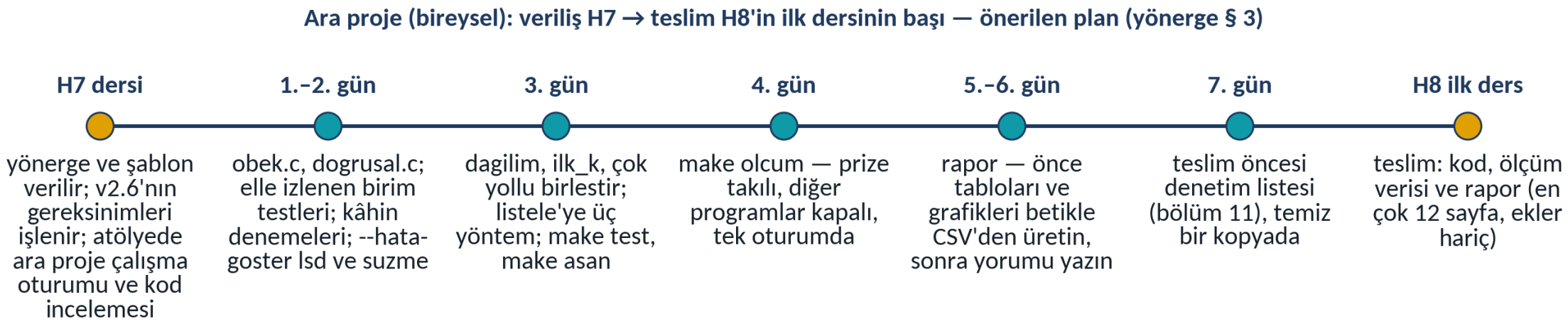
İleri Bakış

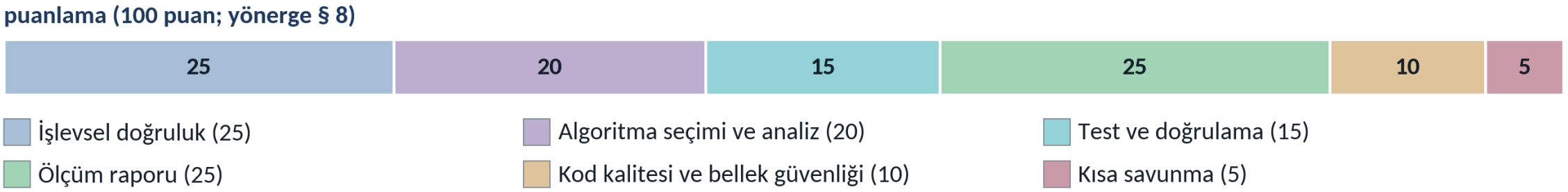
- H8: büyüyen dizi (realloc) ve amortize analiz.
- H12: Dijkstra, Prim, Huffman öncelik kuyruğuyla.
- d-li/binom öbekleri; MSD taban ve sonek dizileri.

Merak: akan ortanca (Örnek 13)

- İki öbek: alt yarı en büyük, üst yarı en küçük öbek.
- $n = 10^5$ akışta sayı başına 23,34 karşılaştırma ($\log_2 n = 16{,}61$); kâhin “evet”.

Ara proje: Mini-OBS 2 v2.6 ve sıralama ölçüm raporu



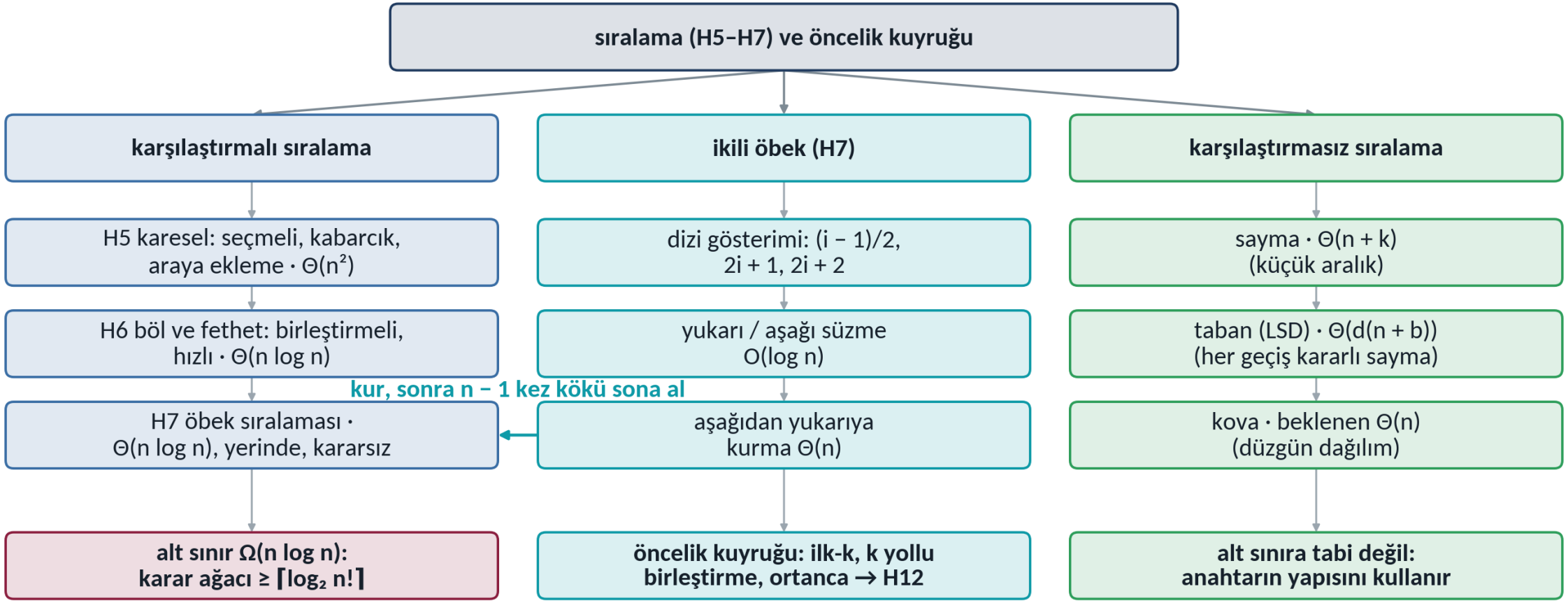


Bireysel · veriliş H7 · teslim H8'in ilk dersinin başı · rapor en çok 12 sayfa (ekler hariç)

Ara proje: v2.6'da ne yazılır, nasıl kabul edilir?

- **Yazılır:** `obek.c` (aşağı süzme, kurma, öbek sıralaması, ilk-k), `dogrusal.c` (kararlı sayma, LSD taban 256), `dagilim`, `ilk_k`, çok yollu `birlestir`; `listele`'ye obek, sayma, taban.
- **Ölçüm raporu:** dokuz yöntem (seçmeli, kabarcık, araya ekleme, birleştirmeli, hızlı, öbek, sayma, taban, qsort) × dört girdi × n; log–log grafik, eğim, kuramla karşılaştırma, seçim rehberi, sınırlılıklar.
- **Kabul (seçme):** Öbek kurma 4 1 3 2 16 9 10 14 8 7 → 16 14 10 8 7 9 3 2 4 1; öbek sıralaması (2,0) (2,1) (1,2) dizisini (1,2) (2,1) (2,0) yapar (kararsız); Taban sıralaması en büyük anahtar 255 iken 1, 256 iken 2 geçiş yapar; 10 haneli öğrenci numaralarında 4 geçiş (olcer kiyas_taban: “geçiş = 4”, “kuram: UYUMLU”).
- **Puan:** işlevsel doğruluk 25 · algoritma seçimi ve analiz 20 · test 15 · ölçüm raporu 25 · kod kalitesi 10 · kısa savunma 5.

Özet: Hafta 7 kavram haritası



Öbek: dizide ağaç, O(log n) süzme, Θ(n) kurma · karşılaştırmalı Ω(n log n) · karşılaştırmasız: koşullu Θ(n)

Vize ve H8'e hazırlık: H1–H7 konu haritası

hafta · başlık ve temel kavramlar (Tablo 4.1)		izleme	karmaşıklık	değişmez	hata bulma	Parsons / tamamlama	strateji seçimi	diller arası okuma
H1	Algoritmalara Dönüş: Doğruluk, döngü değişmezi, kâhinle test, deneysel ölçüm, üç dilli çalışma düzeni	1	·	1	2	1	1	1
H2	Algoritma Analizi: T(n), O/Ω/Θ, büyüme sınıfları, oran testi, Eratosthenes kalburu, hızlı üs alma	1	2	1	1	1	·	1
H3	Özyineleme ile Arama Uzayını Dolaşmak: Özyineleme ağacı, alt küme/permütasyon üretimi, geri izleme, budama	1	1	·	1	1	1	1
H4	Arama Algoritmaları: Doğrusal ve ikili arama, [lo, hi) değişmezi, lower/upper_bound, cevap üzerinde ikili arama	1	1	1	1	·	1	1
H5	Temel Sıralama Algoritmaları: Seçmeli, kabarcık ve araya ekleme sıralaması; ters çevrim, kararlılık, qsort karşılaştırıcısı	1	1	1	1	1	1	1
H6	Böl ve Fethet: Birleştirmeli ve hızlı sıralama, bölümleme, ana teorem, Ω(n log n) alt sınırı, quickselect	1	1	1	1	1	1	1
H7	Sıralamanın Ötesi: İkili öbek, öbek sıralaması, öncelik kuyruğu, sayma ve taban sıralaması	1	1	1	1	1	1	1

Hücre: vize hazırlık soru bankasındaki soru sayısı (toplam 47; Tablo 10.2 soru türleri). Kod yazma türü alıştırmalarda.

Soru bankası web sayfasında (47 soru, hafta ve türe göre) · ara proje teslimi H8'in ilk dersinin başı · H8: dinamik diziler ve bağlı listeler

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Öbek [95 88 92 71 84 63 90 45 52 79]'dan en büyüğü çıkarınca dizi ne olur? Kaç takas?

2. Sayma sıralaması 32 bitlik anahtarlarda neden uygulanamaz? Taban sıralaması bunu nasıl çözer?

3. Taban sıralamasının her geçişi neden kararlı olmalıdır? İki sayıyla bir karşı örnek verin.